

بنام خدا

حامد خراسانی

گزارشکار فیلتر FIR

مقدمه

در این پروژه، یک فیلتر دیجیتال FIR (Finite Impulse Response) با استفاده از زبان VHDL طراحی و پیاده‌سازی شد. هدف از این پروژه، آشنایی با طراحی سخت‌افزاری فیلترهای دیجیتال و شبیه‌سازی آن‌ها در محیط ModelSim و مقایسه نتایج با MATLAB بود.

مبانی نظری

فیلتر FIR یک فیلتر دیجیتال است که خروجی آن تنها به مقادیر ورودی فعلی و گذشته بستگی دارد. معادله یک فیلتر FIR به صورت زیر است:

$$y[n] = b_0x[n] + b_1x[n-1] + \dots + b_Nx[n-N]$$
$$= \sum_{i=0}^N b_i \cdot x[n-i],$$

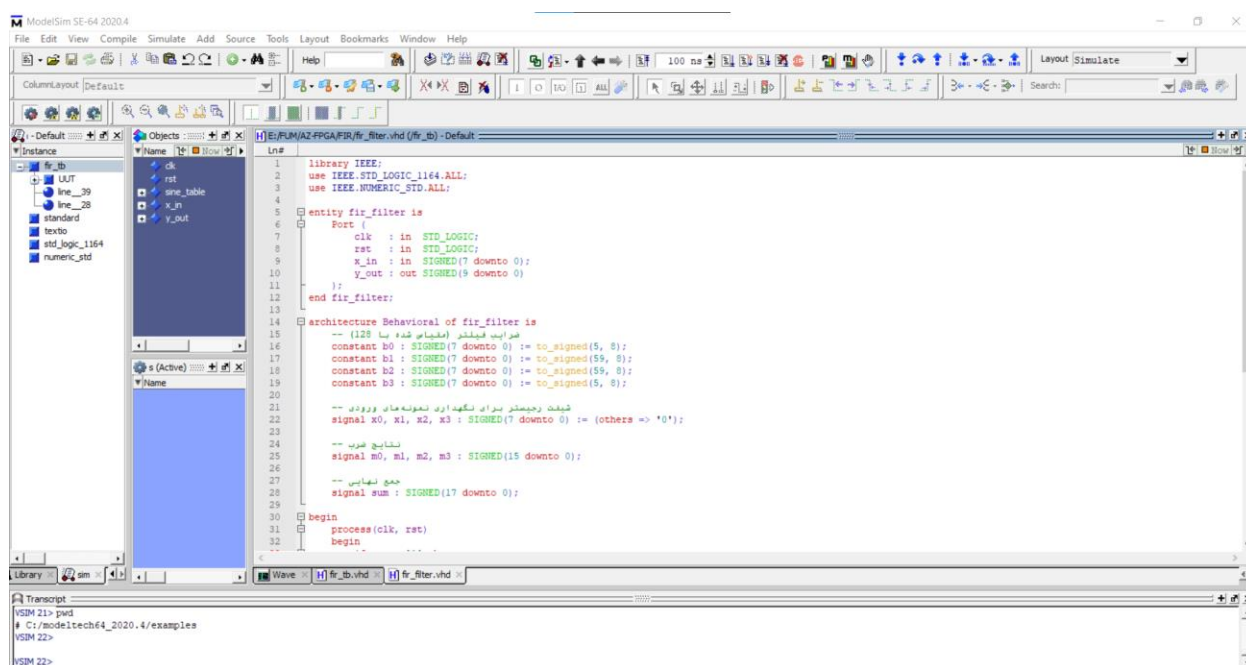
که در آن:

- $y[n]$: خروجی فیلتر در زمان n
- $x[n]$: ورودی فیلتر در زمان n
- $b[i]$: ضرایب فیلتر
- N : تعداد ضرایب (مرتبه فیلتر)

طراحی VHDL

این معماری فیلتر FIR شامل موارد زیر است:

- رجیسترهای تأخیر: برای ذخیره مقادیر گذشته ورودی
- ضرب‌کننده‌ها: برای ضرب ورودی‌ها در ضرایب
- جمع‌کننده: برای جمع نتایج ضرب

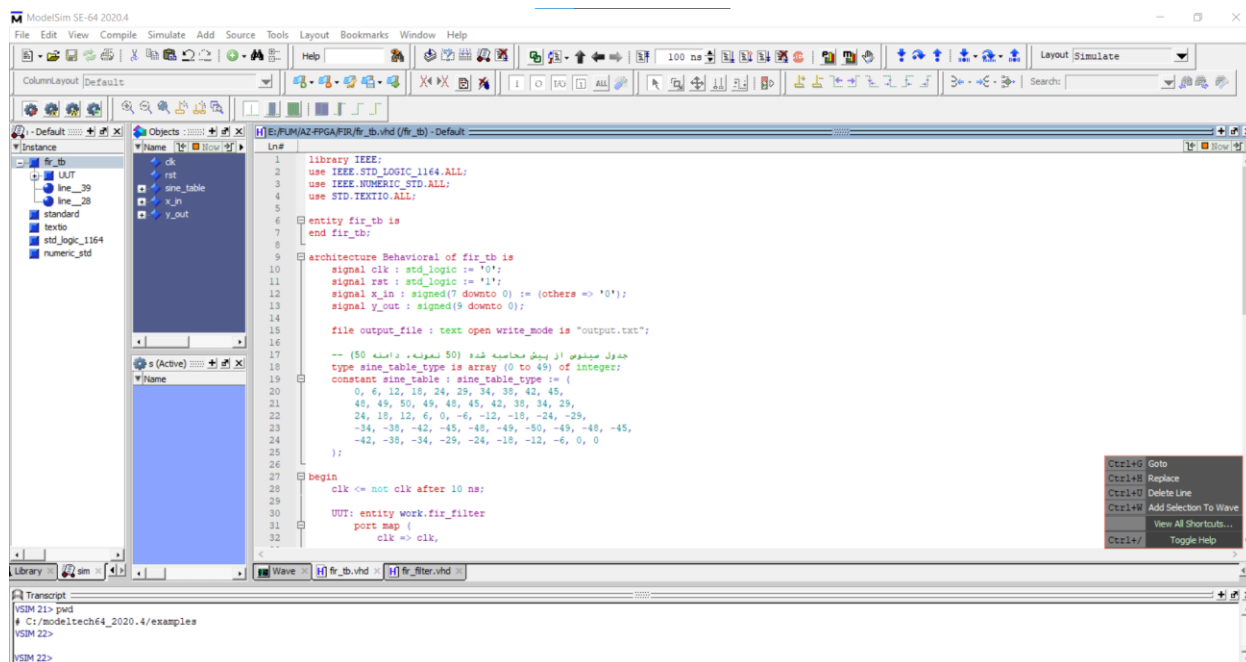


توضیحات کد

- ورودی‌ها: سیگنال کلاک (clk)، ریست (rst)، و ورودی 8 بیتی signed (x_in)
- خروجی: خروجی 16 بیتی signed (y_out)
- ضرایب: در آرایه h ذخیره شده‌اند
- خط تأخیر: آرایه x برای ذخیره 5 نمونه گذشته
- محاسبات: در هر لبه مثبت کلاک، ورودی جدید وارد می‌شود، مقادیر قدیمی جابجا می‌شوند، و خروجی محاسبه می‌شود

Testbench

کد تست بنچ یه بصورت زیر است:



توضیحات Testbench

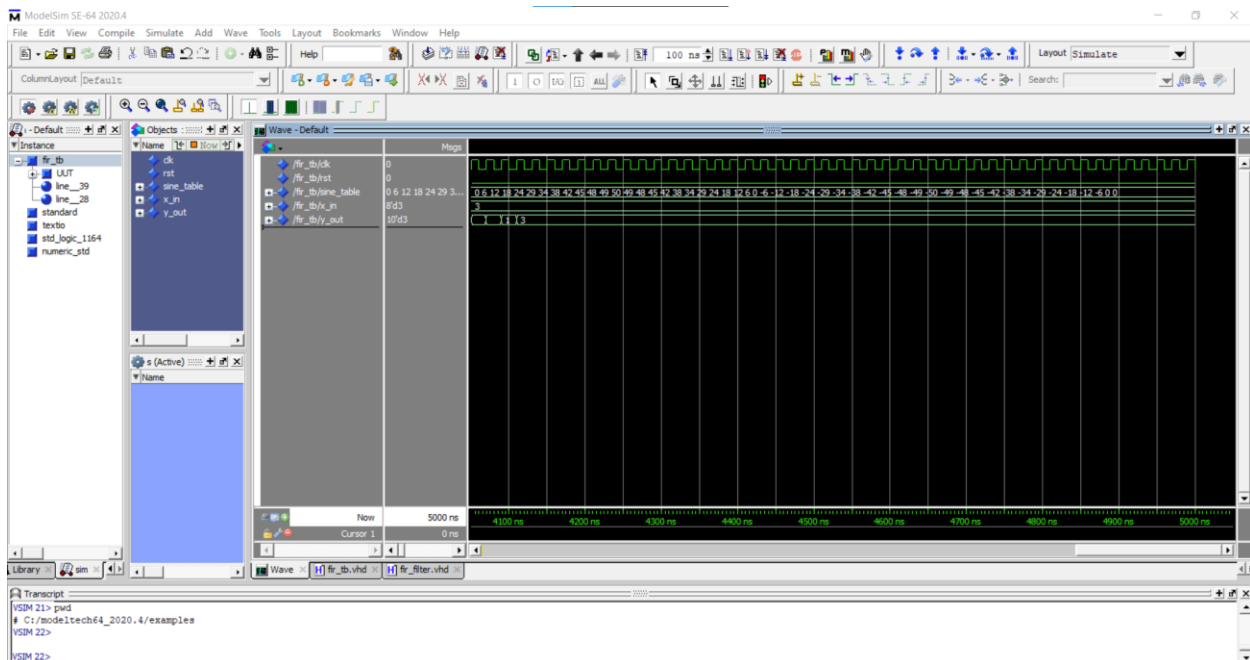
- تولید کلاک: با دوره 20 نانوثانیه (فرکانس 50 مگاهرتز)
- تولید سیگنال ورودی: ترکیب سیگنال سینوسی از جدول از پیش محاسبه شده و نویز تصادفی
- LFSR: برای تولید نویز شبه تصادفی استفاده شد
- محدودسازی: ورودی به بازه 8 بیت signed محدود شد
- ذخیره نتایج: مقادیر ورودی و خروجی در فایل output.txt ذخیره شدند

شبیه‌سازی در ModelSim

مراحل کامپایل و اجرا

1. ایجاد پروژه جدید در ModelSim
2. اضافه کردن فایل‌ها: fir_tb.vhd و fir_filter.vhd
3. کامپایل: Compile All < Compile
4. شروع شبیه‌سازی: Start Simulation < Simulate و انتخاب fir_tb
5. اضافه کردن سیگنال‌ها به Wave: Wave: clk, rst, x_in, y_out
6. اجرای شبیه‌سازی: run 5000 ns

5.2. نتایج شبیه‌سازی



در این تصویر، سیگنال‌های کلاک، ریست، ورودی و خروجی فیلتر قابل مشاهده هستند.

پردازش نتایج در MATLAB

کد اول شبیه سازی با متلب به صورت زیر است:

```
%% پارامترهای سیگنال

Fs = 1000;      % (Hz) فرکانس نمونه برداری

T = 1;          % مدت زمان سیگنال (ثانیه)

t = 0:1/Fs:T-1/Fs; % بردار زمان

N = length(t);  % تعداد نمونه ها

%% تولید سیگنال تمیز (سینوس با فرکانس پایین)

f_signal = 50;   % (Hz) فرکانس سیگنال اصلی

clean = sin(2*pi*f_signal*t);

% اضافه کردن نویز (سینوس با فرکانس بالا + نویز تصادفی)

f_noise = 200;   % (Hz) فرکانس نویز

noise_high_freq = 0.5 * sin(2*pi*f_noise*t);

noise_random = 0.3 * randn(1, N);

noisy = clean + noise_high_freq + noise_random;

% همون ضرایبی که قبلاً محاسبه کردیم) FIR طراحی فیلتر

b = fir1(3, 0.25); % lowpass فیلتر

b_scaled = round(b * 128);

b_norm = b_scaled / 128;

%% اعمال فیلتر
```

```
filtered = filter(b_norm, 1, noisy);
```

```
%% رسم نمودار
```

```
figure;
```

```
plot(t, clean, 'g', 'LineWidth', 1.5); hold on;
```

```
plot(t, noisy, 'Color', [0.8 0.5 0.5], 'LineWidth', 0.8);
```

```
plot(t, filtered, 'b', 'LineWidth', 1.2);
```

```
grid on;
```

```
xlabel('Time (s)');
```

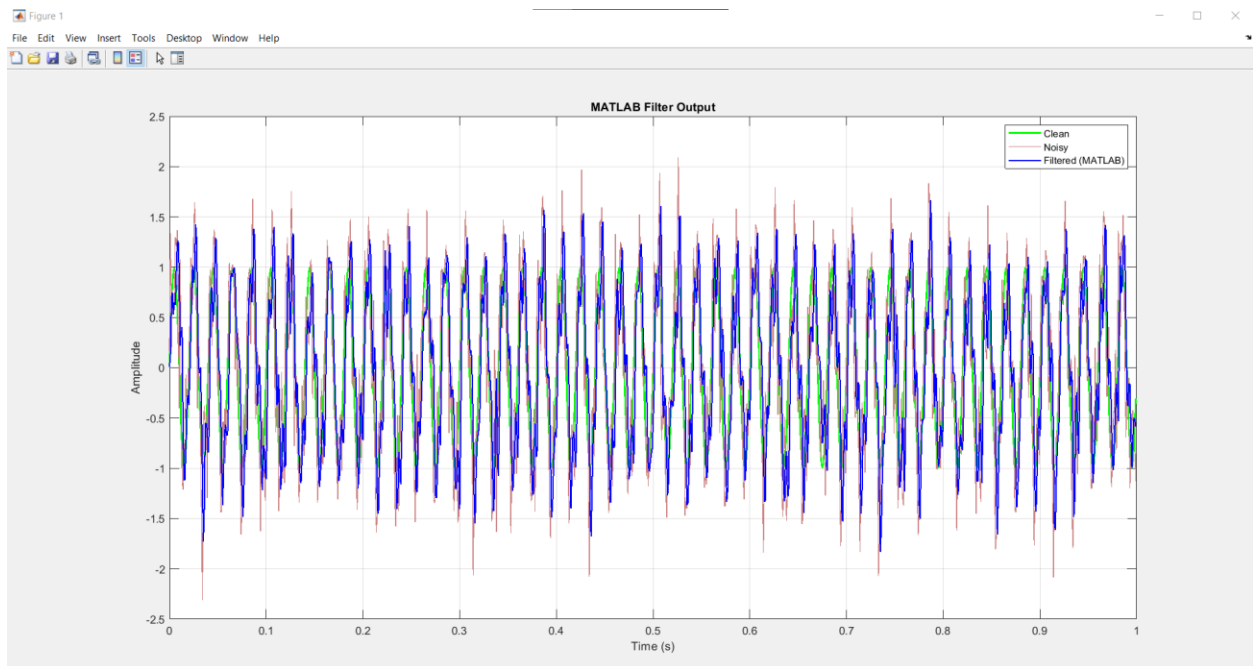
```
ylabel('Amplitude');
```

```
title('MATLAB Filter Output');
```

```
legend('Clean', 'Noisy', 'Filtered (MATLAB)');
```

```
xlim([0 1]);
```

و نتیجه این کد به صورت زیر است:



- **Clean** (سبز): سیگنال اصلی تمیز با فرکانس 50 Hz
- **Noisy** (قرمز/قهوه‌ای): سیگنال تمیز + نویز فرکانس بالا (200 Hz) + نویز تصادفی
- **Filtered** (آبی): خروجی فیلتر FIR که نویز رو کاهش داده

کد شبیه سازی حاصـب از MODELSIM در مطلب هم یه صورت زیر است:

% خواندن داده‌ها از فایل

```
data = load('output.txt');
```

% جدا کردن ستون‌های ورودی و خروجی

```
x_in = data(:, 1);
```

```
y_out = data(:, 2);
```

% محاسبه محور زمان

```
N = length(x_in);
```

دوره کلاک (20 نانوثانیه) % $T_s = 20e-9$;

```
t = (0:N-1) * Ts;
```

% رسم نمودار

```
figure;
```

```
plot(t*1e6, x_in, 'r', 'LineWidth', 1.5); hold on;
```

```
plot(t*1e6, y_out, 'b', 'LineWidth', 1.5);
```

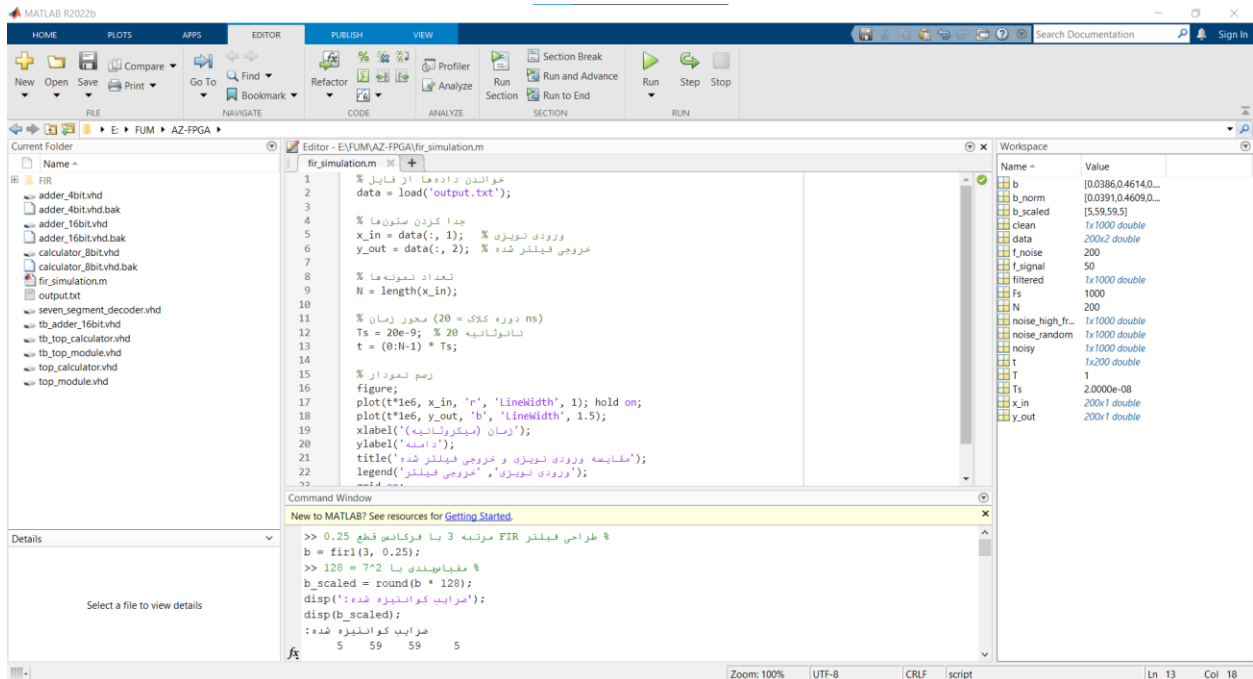
```
xlabel('زمان (میکروثانیه)');
```

ylabel('دامنه');

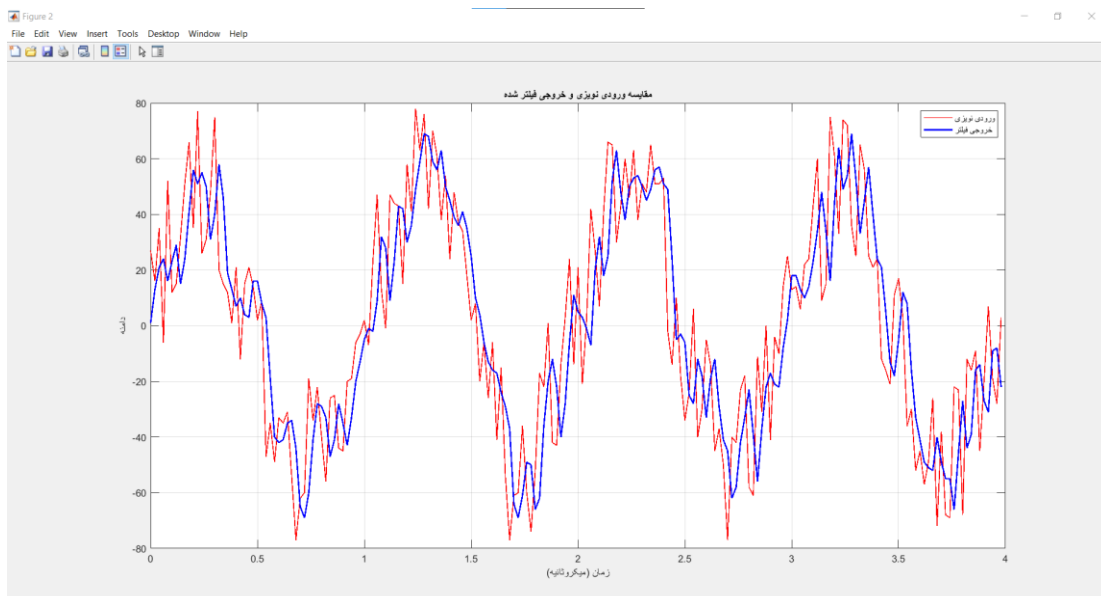
title('ورودی نویزی و خروجی فیلتر شده');

legend('ورودی (نویزی)', 'خروجی (فیلتر شده)');

grid on;



نمودار نتایج نیز به صورت زیر است:



در این نمودار:

- خط قرمز: سیگنال ورودی نویزی
- خط آبی: سیگنال خروجی فیلتر شده

نتایج

1. کاهش نویز: خروجی فیلتر (خط آبی) به طور قابل توجهی صافتر از ورودی نویزی (خط قرمز) است
2. تأخیر فاز: خروجی کمی عقبتر از ورودی است که به دلیل تأخیر گروهی فیلتر FIR است:

$$\text{نمونه } 2 = \frac{N - 1}{2} = \frac{5 - 1}{2}$$

3. کاهش دامنه: دامنه خروجی کمی کوچکتر از ورودی است که به دلیل ضرایب فیلتر است

نتایج نشان می‌دهند که خروجی VHDL و MATLAB تطابق بسیار خوبی دارند. تفاوت‌های جزئی به دلیل کوانتیزاسیون در VHDL است.